Korea Technology and Information Promotion Agency for SMEs

(max 10 pages)

Technology Proposal for 2026 TIPA* International Joint R&D Program

(TIPA: Korea Technology and Information Promotion Agency for SMEs)

The objective of this Technology Proposal is to introduce the institute's proprietary technologies and present potential collaborative areas for Korean SMEs, facilitating joint research and development projects under the 2026 TIPA International Joint R&D Program. This Technology Proposal will be publicly disclosed in conjunction with the official announcement of the 2026 TIPA International Joint R&D Program.

1. Technology Proposer Information

Proposer	**Name**	André Gilerson	**Email**	andre.gilerson@ipt.fraunhofer.de
Institute Name	Fraunhofer Institute for Production Technology IPT			
Address	Steinbachstraße 17, 52074 Aachen, Germany			

2. Technology Overview

Title of Proposed Technology	Enabling critical reliability in AI coding agents by leveraging formal code verification			
Korea's 12 National Strategic Technology Fields	Semiconductor/Display (), Secondary Battery ( ), Advanced Mobility ( ), Next-gen Nuclear Power ( ), Advanced Bio ( ), Aerospace/Marine ( ), Hydrogen ( ), Cybersecurity ( ), AI (√), Next-gen Communication ( ), Advanced Robotics & Manufacturing (), Quantum ( )			
Industrial Technology Classification [1]	**Technology Field 1** (Large Category)		**Technology Field 2** (Large Category)	
	Knowledge Services		Knowledge Services	
	Medium Category	Small Category	Medium Category	Small Category
	R&D / Engineering Services	*Other R&D / Engineering-Related Technologies*	*R&D / Engineering Services*	*Design Information Integration Management / Collaboration System Performance Enhancement Technologies*
Concept of Proposed Technology	Current AI coding agents, primarily driven by Large Language Models (LLMs), have demonstrated remarkable capabilities in code generation but suffer from a fundamental "black box" reliability issue. Because these models operate probabilistically, they frequently produce code that appears syntactically correct but contains subtle logic errors, security vulnerabilities, or hallucinations that evade detection by human reviewers. While Formal Verification, the mathematical proof of software correctness, is an effective system for eliminating such errors, it is currently restricted to high-stakes sectors like aerospace. This restriction exists because traditional formal methods differ vastly from standard coding practices, requiring specialized mathematical expertise and labor-intensive manual effort that makes them prohibitively expensive for general software development.			

	This project proposes the development of a Proof-Guided Synthesis framework integrating it as an automated step in the AI code generation pipeline. Rather than generating raw code directly from a prompt, our tooling will first instruct the agent to generate a formal specification and a corresponding mathematical proof (e.g., using theorem provers like Lean or Rocq), which validates the system's logic against the user's intent. Once the user reviews and accepts this high-level verification, the system uses the proven specification to synthesize the actual executable code. This approach effectively solves the alignment problem by ensuring the final software is mathematically guaranteed to meet its specifications, transforming formal verification from a manual bottleneck into an automated, accessible safeguard for everyday software engineering.
Rationale for Joint R&D Project	Germany provides expertise in Trustworthy AI systems (driven by regulatory frameworks such as EU AI Act), and the deep theoretical foundation for safety critical software systems. The South Korean R&D landscape is characterized by a dynamic SME sector, world-leading ICT infrastructure, and rapid consumer adoption of AI technologies, offering an ideal environment for iterative prototyping and user-centric product design. This project leverages the German capabilities in complex mathematical proof systems with the Korean SME's agility in software tooling and rapid market deployment
Key Words	1. Formal Verification 2. Generative AI 3. AI Coding Agents 4. Trustworthy AI

3. Research and Development Objectives & Plan

Technology Readiness Level (TRL)[2]	Start of the Project	Level (3)	End of the Project	Level (6)
R&D Objectives	The R&D effort will focus on bridging the gap between abstract formal methods and practical software engineering, transitioning from isolated theoretical models to an integrated toolchain that allows developers to utilize mathematical verification without requiring specialized expertise. Specifically, the project aims to achieve the following: • **Develop a Fromal Verification Engine:** Engineer a specialized AI module capable of reliably translating natural language user intent into formal logic specifications (e.g., using Lean or Rocq) and auto generating the corresponding mathematical proofs. • **Establish a "Correct-by-Construction" Pipeline:** Create a synthesis mechanism that derives executable code (such as Python or C++) directly from the verified specifications, ensuring the final output is mathematically guaranteed to be free of hallucinations and logic errors found in the proof. • **Design a Human-Centric Verification Interface:** Build an intuitive user interface that abstracts the complexity of theorem proving, allowing standard developers to review and validate high-level system logic rather than debugging generated code. • **Validate Efficacy in Operational Environment**: Deploy the prototype in a pilot software project to quantitatively demonstrate a reduction in critical bug density and review time compared to standard LLM-based coding agents.			
R&D Target Specifications	• Achieve a success rate of >90% for generating syntactically valid formal specifications (e.g., in Lean/Rocq) from natural language prompts on the first pass for single-function algorithmic modules (up to 50 lines of code). • The system shall automatically discharge >50% of generated proof obligations without human intervention for standard data structure manipulations, reducing the manual burden on the human reviewer. • Demonstrate a measured reduction of logic errors compared to standard unverified LLM code generation when tested against a standard coding benchmark (e.g., a subset of HumanEval or MBPP). • Enable software engineers without formal methods training to successfully verify and generate code for low-complexity modules • Achieve a low "spec-to-verified-code" cycle time for standard algorithmic functions, ensuring the tool is responsive enough for iterative development workflows.			

R&D Plan	Year 1: Foundation & "The Neurosymbolic Engine"
	
	Focus: Developing the core technology that translates natural language into verifiable math specifications.
	
	• WP1: Dataset & Corpus Creation
	  ○ Curate a specialized training dataset of parallel "Code - Formal Spec" pairs (e.g., Python code mapped to Lean/Rocq theorems).
	  ○ Define the formal logic subset (DSL) required for the specific target domain (e.g., algorithmic logic, data structures).
	• WP2: Translation Model Development
	  ○ Train/Fine-tune a "Translator Agent" (LLM) to convert natural language requirements into formal specifications.
	  ○ Implement "Syntax-Guided Decoding" to ensure the agent only outputs syntactically valid proof code.
	• WP3: Proof Automation			

- Build a bridge to an automated theorem prover (e.g., Lean 4) that attempts to verify the generated specs automatically.

- WP4: Initial UI Mockups
  - Design the user experience (UX) for how a non-expert developer will interact with a "mathematical specification" without needing to read the math.

Year 2: Synthesis Pipeline & "Correct-by-Construction"

Focus: Connecting the verified specs to actual code generation and testing in a lab environment.

- WP5: Spec-to-Code Synthesis
  - Develop the "Compiler Agent" that takes the *verified* specification and generates executable code (Python/C++).
  - Create a feedback loop where the proof engine rejects code that violates the spec.
- WP6: IDE Integration
  - Develop a VS Code or JetBrains plugin that houses the tool.
  - Implement the "Human-in-the-Loop" review interface where users approve the high-level intent before the proof engine runs.
- WP7: Laboratory Validation
  - Test the full pipeline on a "Golden Set" of known coding problems (e.g., HumanEval benchmark) to measure hallucination reduction.
  - Optimize the latency of the proof generation (aiming for <2 mins).

Year 3: Pilot & Deployment

Focus: Moving from the lab to a real-world developer workflow.

- WP8: Pilot Integration
  - Deploy the alpha version of the tool within the SME's own development teams for internal use.
  - Collect telemetry data on usage patterns, crash rates, and user friction points.
- WP9: Optimization & Hardening
  - Refine the theorem prover heuristics based on Year 3 failure cases.
  - Expand the supported library of specifications to cover more complex software modules (e.g., API calls, state management).
- WP10: Validation & Commercial Roadmap
  - Quantify the reduction in bug density compared to standard tools (Final TRL 6 Demonstration).
  - Prepare the documentation and licensing strategy for commercial launch or open-source release.

4. R&R (Roles and Responsibilities)

Expected R&R of Research Institute	• Architect the core engine to translate natural language into formal logic (e.g., Lean /Coq). • Develop heuristics to automate theorem proving and minimize manual intervention. • Fine-tune models to constrain outputs to "valid-by-construction" standards. • Benchmark the system to quantify error reduction and verify mathematical correctness. • Ensure compliance with EU AI Act and safety regulations.
Expected R&R of Korean SME Partner	• Develop plugins (e.g., for VS Code) to embed the tool into standard coding environments. • Create an accessible interface that hides mathematical complexity from non-expert users. • Manage cloud backend, API orchestration, and system performance. • Deploy the prototype (TRL 6) internally to gather real-world usage data. • Tailor the product for commercial adoption and agile workflows.

5. Expected Outcomes/Impact

Expected Outcomes and Impact	This technology will enable developers to build critical safety-grade reliability into everyday applications without requiring specialized mathematical expertise. Economically, this shifts the software engineering paradigm from expensive post-hoc debugging to a cost-efficient correct-by-construction model, projected to significantly reduce quality assurance overhead and time-to-market for critical systems. This entails a critical innovation in AI coding agents, opening a large market currently dominated by American companies. Strategically, this collaboration positions the German-Korean consortium as a pioneer in Trustworthy AI.

☐ Terms:

1) Industrial Technology Classification

· Classification system according to Industrial Technology Classification Table (**Appendix 1**, based on Article 16 of the Common Operational Guidelines for the Industrial Technology Innovation Project in Korea).

2) Technology Readiness Level (TRL)

Description		
·A consistent and objective indicator of the technological maturity of key element technologies		
	Basic Research Stage (1-2)	1st Stage: Basic theories and experiments 2nd Stage: Proof of Concept, such as practical ideas or patents
	Experimental Stage (3-4)	3rd Stage: Verification of basic performance at a laboratory scale 4th Stage: Evaluation of key performance of materials, components, and systems at a laboratory scale
	Prototype Stage (5-6)	5th Stage: Production and performance evaluation of prototypes of finalized materials, components, and systems 6th Stage: Production and performance evaluation of prototypes at a pilot scale
	Productization Stage (7-8)	7th Stage: Reliability testing and evaluation by potential customers 8th Stage: Prototype certification and standardization
	Commercialization Stage (9)	9th Stage: Commercialization